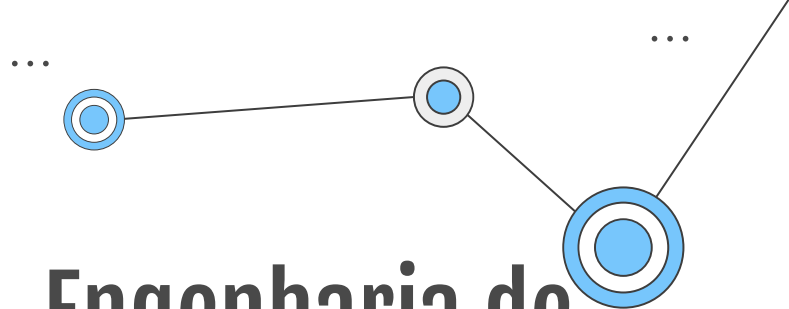
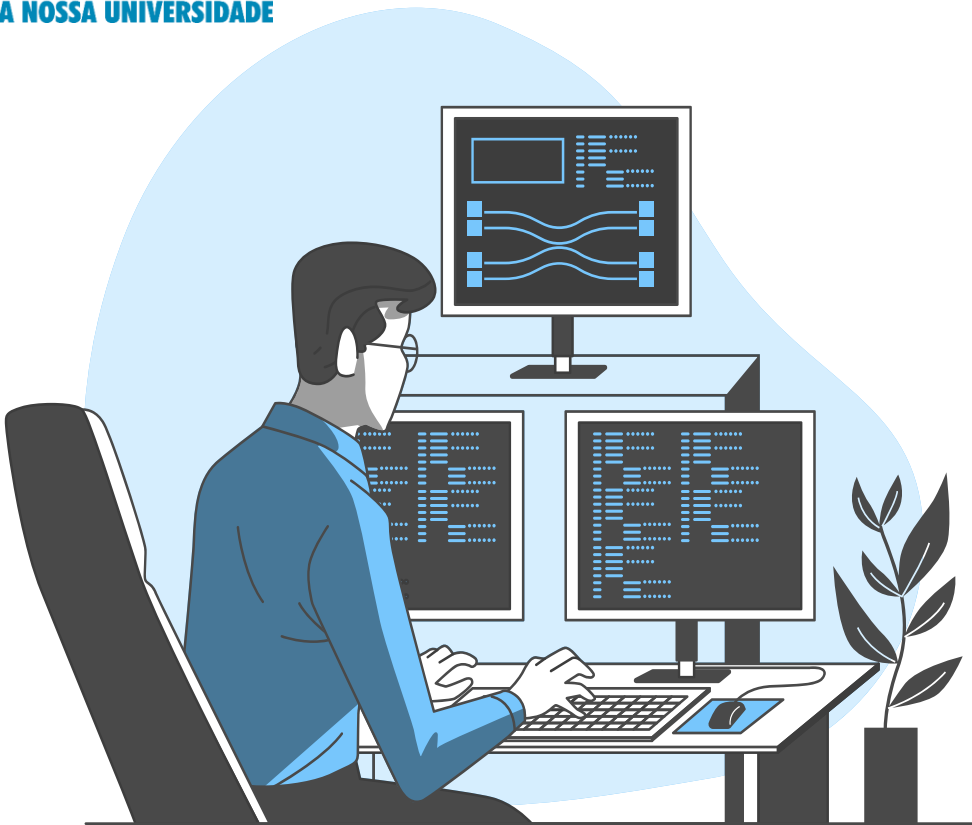




A NOSSA UNIVERSIDADE



Engenharia de.. Requisitos

**Conceitos Fundamentais:
Requisitos, Tipos e Falhas
Recorrentes**

Aula 2
Prof. Me. Daniel Cunha da Silva

Nessa aula

01
...

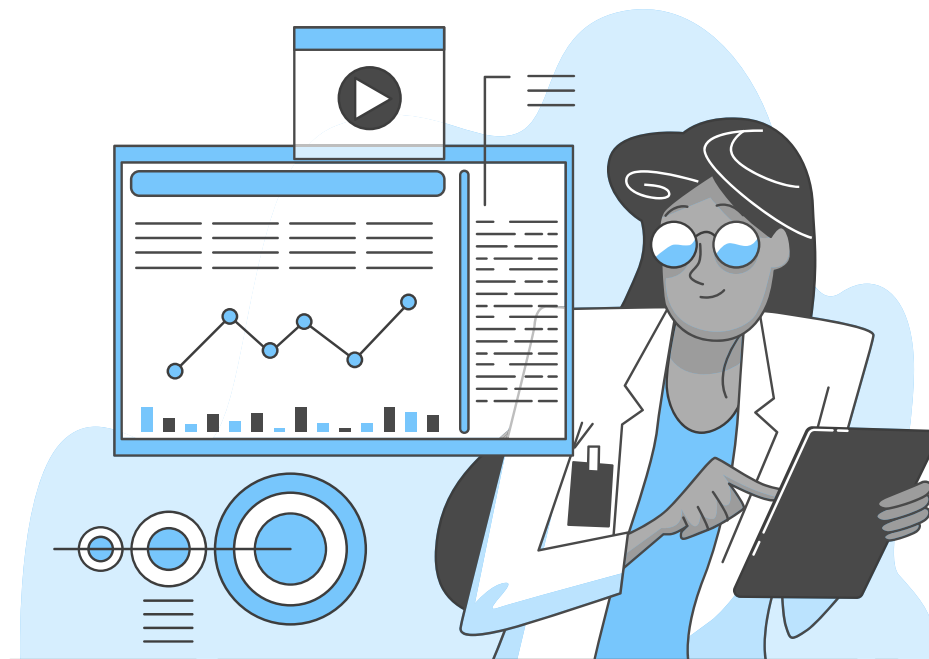
Requisitos

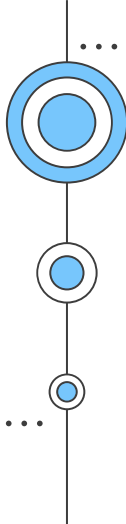
02
...

Tipos de Requisitos

03
...

Falhas Recorrentes



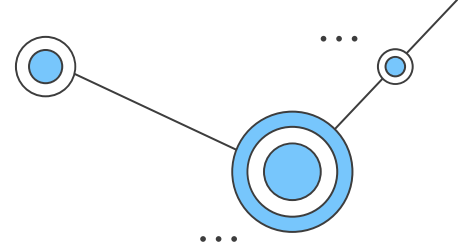


01

Requisitos



Definição de Engenharia de Requisitos

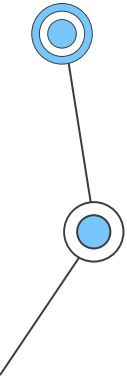


O que são Requisitos?

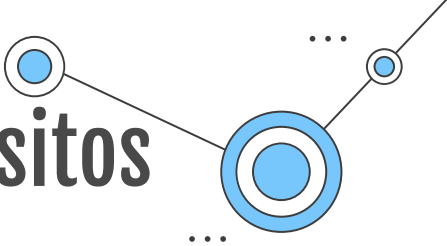
- Definem o que um sistema deve fazer e sob quais restrições (*Aula anterior*).
- Uma condição ou capacidade que deve ser atendida ou possuída por um sistema ou componente de sistema para satisfazer um contrato, norma, especificação ou outro documento formalmente imposto. (*Definição padrão da IEEE*).

O que é Engenharia de Requisitos?

- É o nome que se dá ao conjunto de atividades relacionadas com a descoberta, análise, especificação e manutenção dos requisitos de um sistema (*Aula anterior em conformidade com a definição atual dada por Sommerville*).



Linha do Tempo da Engenharia de Requisitos



1968–1979 — A Crise do Software e a Origem da Disciplina

- Modelo Cascata
- Surgimento do SRS (*Software Requirements Specification*)
- IEEE Std 610.12:1990 (hoje, a ISO/IEC/IEEE 24765:2017)
- Ênfase documental
- Requisitos como fase inicial
- Processo sequencial

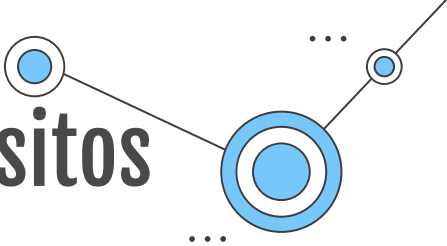
1990–1999 — Consolidação Científica

- 
- Relatório da *NATO Science Committee* (1968)
 - Sistemas militares e industriais falhando
 - Atrasos e estouro de orçamento
 - Pouca formalização
 - Foco técnico excessivo
 - Usuários pouco envolvidos

1980–1989 — Formalização e Documentação

- Conferências específicas de ER
- Abordagens *Goal-Oriented* (KAOS, i*)
- Modelagem por Casos de Uso
- ER passa a ser vista como disciplina própria
- Inclusão de: Elicitação, Negociação e Validação

Linha do Tempo da Engenharia de Requisitos



2000–2009 — Agilidade e Mudança Contínua

- Integração contínua
- Entrega contínua
- Monitoramento em produção
- Sistemas distribuídos
- Considerar atributos de qualidade críticos
- Surge a noção de Engenharia de Requisitos Contínua

2020— Atualidade — Transformação Digital e IA

2010–2019 — DevOps e Sistemas Sociotécnicos

- Requisitos iterativos
- *User Stories* substituindo SRS extensos
- Validação frequente
- Prototipação
- Requisitos tornam-se dinâmicos

- Sistemas baseados em IA
- Ecossistemas interconectados
- Regulamentações complexas
- Sistemas adaptativos
- Uso de IA para análise semântica
- Modelagem baseada em conhecimento (ontologias)
- Integração com arquitetura e MBSE

Linha do Tempo da Engenharia de Requisitos

Período	Natureza da ER	Visão da Mudança	Papel Estratégico
1970	Implícita	Indesejada	Baixo
1980	Documental	Controlada	Médio
1990	Disciplina científica	Negociável	Médio-Alto
2000	Iterativa	Esperada	Alto
2010	Contínua	Estrutural	Muito Alto
2020+	Estratégica e adaptativa	Permanente	Central

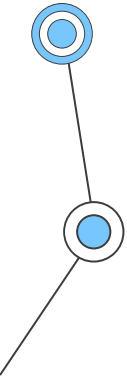
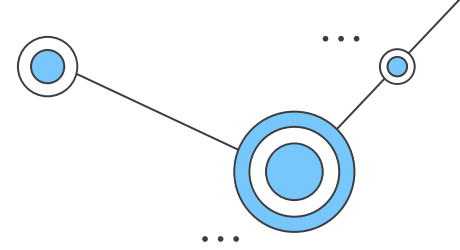


02

Tipos de Requisitos



Tipos de Requisitos

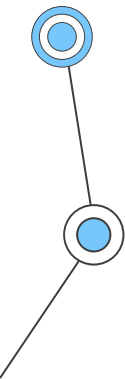
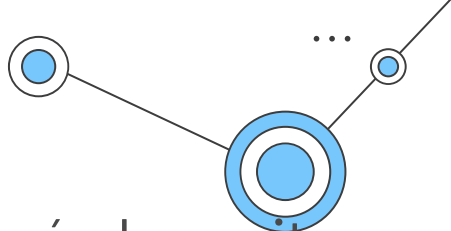


Por Abstração

Requisitos de usuários são requisitos de mais alto nível, escritos por usuários, normalmente em linguagem natural e sem entrar em detalhes técnicos.

Requisitos de sistema são técnicos, precisos e escritos pelos próprios desenvolvedores. Daqui ramifica-se em funcionais e não funcionais.

Requisitos de usuário estão mais próximos do problema, enquanto que requisitos de sistema estão mais próximos da solução.



Por Natureza

REQUISITOS NÃO FUNCIONAIS



REQUISITOS FUNCIONAIS QUE COMPÕEM AS CARACTERÍSTICAS QUE O SISTEMA DEVE RESPEITAR AO ATENDER CADA UM DOS REQUISITOS FUNCIONAIS SOLICITADOS

Análise de REQUISITOS

Fonte: <https://analisederequisitos.com.br/requisitos-funcionais-e-nao-funcionais/>

Requisitos funcionais são todas as necessidades, características ou funcionalidades esperadas em um processo que podem ser atendidos pelo software. expressa uma ação que deve ser realizada através do sistema, ou seja, um requisito funcional é “**o que sistema DEVE fazer**”.

Requisitos não funcionais podem ser definidos como “**DE QUAL MANEIRA o sistema deve fazer**”. Eles devem sempre ser mensuráveis, ou seja, deve ser possível verificar se ele está ou não sendo atendido pelo software.

Por Natureza

É necessário lembrar que um **requisito não funcional** deve ser SEMPRE verificável (medido, aferido ou mensurado), caso contrário ele não pode ser interpretado como tal.

Alguns exemplos de requisitos que não representam requisitos não funcionais:

- O sistema deve ser moderno
- O sistema deve ser rápido
- O programa deve ser seguro



Exemplo



Considere um sistema de **Aplicativo de Entrega de Comida**.

Foi exigido que o sistema tivesse as seguintes características:

- Permitir que o usuário entre no sistema com e-mail e senha.
- O usuário pode ver restaurantes e itens disponíveis.
- O usuário seleciona itens, adiciona ao carrinho e finaliza o pedido.
- O sistema deve processar o pagamento e confirmar a transação.
- O sistema precisa mostrar o status do pedido em tempo real.
- Deve possuir uma interface clara e intuitiva.
- A interface deve responder em $\leq 2s$.
- É preciso suportar até 50.000 usuários simultâneos.
- Dados e pagamentos devem ser criptografados.
- Funcionamento em Android e iOS.

**Requisitos
funcionais**

**Requisitos
não
funcionais**

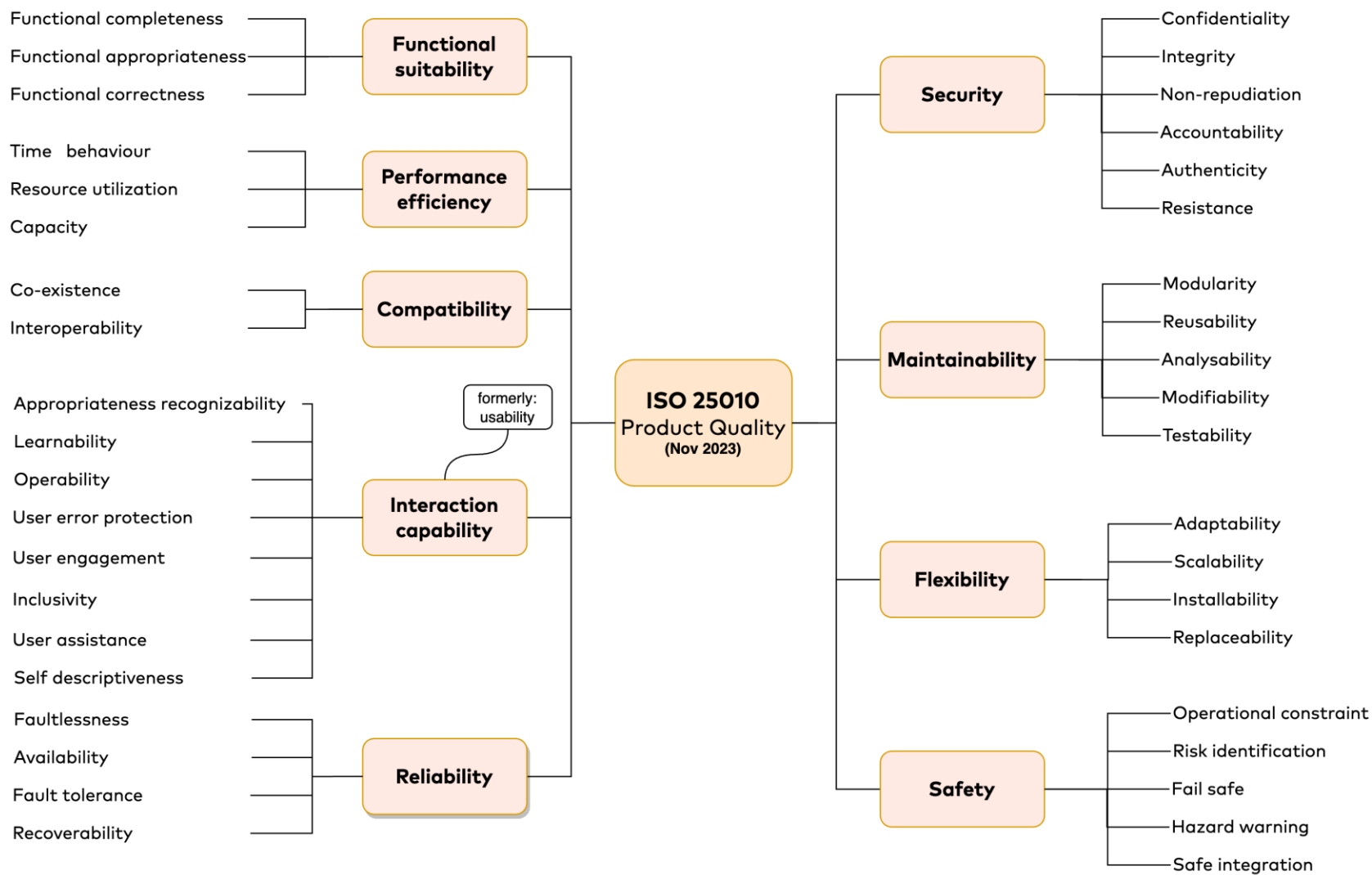
ISO/IEC 25010

É uma norma internacional da família SQuaRE (*Systems and Software Quality Requirements and Evaluation*) que define um modelo estruturado para avaliar a qualidade de produtos de software e sistemas.

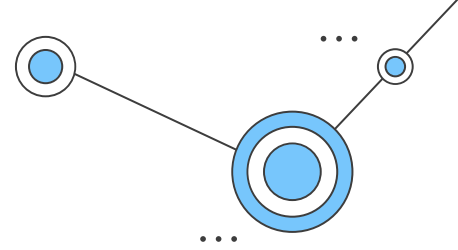
Estabelece 8 características principais: funcionalidade, eficiência, compatibilidade, usabilidade, confiabilidade, segurança, manutenibilidade e portabilidade.

Características:

- Não define requisitos diretamente.
- Define características de qualidade.
- Fornece base formal para especificar RNFs.
- Ajuda a tornar RNFs mensuráveis e verificáveis.



Por Origem

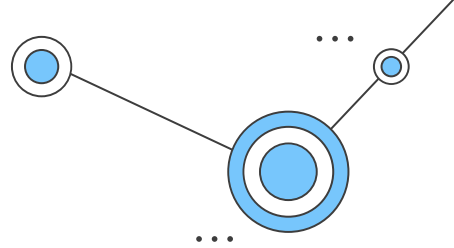


Requisitos de negócio

- De acordo com a ISO 29148, são declarações que descrevem objetivos organizacionais, metas estratégicas, necessidades operacionais ou problemas que justificam o desenvolvimento de um sistema, estabelecendo o valor esperado para a organização.
- De acordo com Karl Wieggers, os requisitos de negócio descrevem os objetivos de alto nível da organização ou do cliente que solicita o sistema.
- Dois elementos principais dos requisitos de negócios são a **visão** e o **escopo**.
 - A visão do produto descreve de forma sucinta o produto que alcançará os objetivos de negócio.
 - O escopo do projeto identifica qual porção da visão do produto está em desenvolvimento.

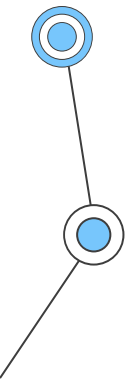


Por Origem

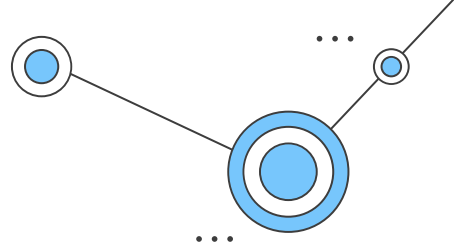


Requisitos de negócio

- Quando se definem os requisitos de negócio temos comumente as seguintes informações:
 - O contexto do problema (background)
 - As oportunidades de negócio
 - Objetivos de negócio (de forma quantitativa e mensurável)
 - Ex: *Aumentar a rastreabilidade de todas as transações bancárias em 40% em 3 meses.*
 - A visão do produto
 - Principais funcionalidades
 - Limitações
 - Regras de negócio
 - Diagrama de Contexto

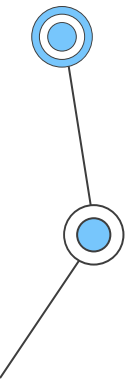


Por Origem

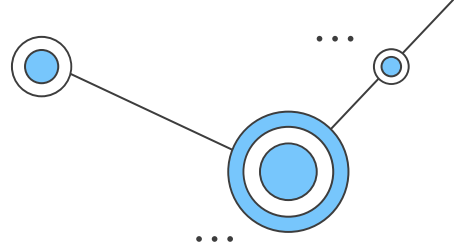


Requisitos de negócio

- Quanto a visão do produto, ela deve ser sucinta de forma a englobar a proposta e objetivo do produto em um parágrafo.
- **Template da Visão de Produto:**
 - Para: <cliente>
 - que: <necessidade ou oportunidade>
 - o: <nome do produto>
 - é: <categoria do produto>
 - que: <funcionalidades>
 - de modo contrário: <alternativas, competidores>
 - nosso produto: <diferenças e vantagens do novo produto>



Por Origem

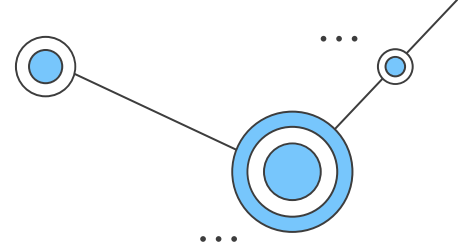


Requisitos de negócio

- Exemplo de visão de produto:
 - **Para** clínicas médicas de pequeno e médio porte que precisam reduzir faltas e otimizar a agenda, **o** CliniSmart **é** um sistema inteligente de gestão e agendamento médico **que** permite agendamento online, confirmação automática via WhatsApp e previsão de faltas com base em histórico. **De modo contrário** a planilhas manuais e sistemas tradicionais sem inteligência analítica, **nosso produto** oferece recomendações inteligentes e painel gerencial em tempo real para apoiar decisões estratégicas.



Por Origem

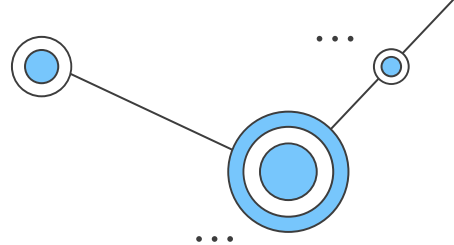


Requisitos regulatórios

- De acordo com a ISO 29148, são declarações que impõem obrigações ao sistema derivadas de leis, normas técnicas, regulamentos governamentais ou contratos formalmente vinculativos, devendo ser obrigatoriamente atendidas para garantir conformidade legal ou institucional.
- Podem entrar como objetivos no documento de requisitos de negócio.
- Exemplo:
 - **O sistema deve armazenar dados pessoais em conformidade com a Lei Geral de Proteção de Dados (LGPD).**
 - **O sistema bancário deve atender às resoluções do Banco Central.**
 - **O sistema médico deve seguir padrões HL7 para interoperabilidade.**



Por Obrigatoriedade



Consiste na categorização dos requisitos de acordo com seu grau de essencialidade para que o sistema entregue valor mínimo aceitável, atenda a compromissos contratuais ou satisfaça restrições legais.

Um dos modelos mais consolidados é o método **MoSCoW**, amplamente utilizado em contextos ágeis.



Must have

Deve ter

Aquilo que é considerado obrigatório ou imprescindível para o projeto ou negócio.



Should have

Deveria ter

Tudo aquilo que é importante ter, mas não é imprescindível para o projeto ou negócio.



Could have

Poderia ter

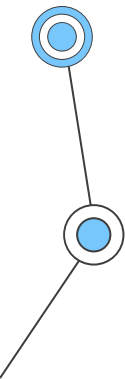
Tudo aquilo que não é essencial, mas seria bom ter ou poderia ser um diferencial.



Won't have

Não terá

Não agrega valor ao negócio no momento e por enquanto não será feito.



03

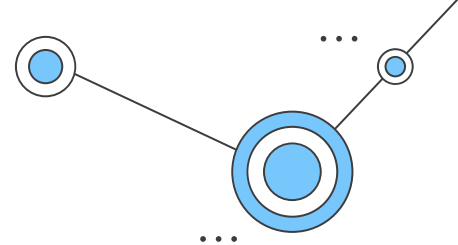
Falhas Recorrentes

Armadilhas no Processo de ER



- **Requisitos vagos ou ambíguos:** Requisitos mal definidos levam a mal-entendidos e entregas desalinhadas.
- **Scope Creep:** Alterações descontroladas nos requisitos podem inviabilizar projetos, aumentando custos e prazos.
- **Envolvimento inadequado das partes interessadas:** Engajamento insuficiente resulta em requisitos incompletos ou irrelevantes.
- **Falta de rastreabilidade:** Dificuldade em rastrear requisitos em todo o ciclo de vida da engenharia de requisitos pode levar a inconsistências e problemas de conformidade.
- **Resistência à Mudança:** As equipes geralmente têm dificuldades para se adaptar às exigências em evolução, principalmente em ambientes ágeis.

Falhas Recorrentes



A maioria das falhas em projetos de software não decorre de erro técnico de implementação, mas de problemas na definição, análise e gestão dos requisitos.

Ambiguidade

- Um requisito é ambíguo quando permite múltiplas interpretações válidas.
- “O sistema deve ser rápido”

Implementações divergentes;
Conflitos entre cliente e equipe técnica;
Retrabalho.

Incompletude

- Ocorre quando o requisito não contempla todos os cenários necessários para sua execução.
- “O sistema deve permitir cadastro de usuário.”

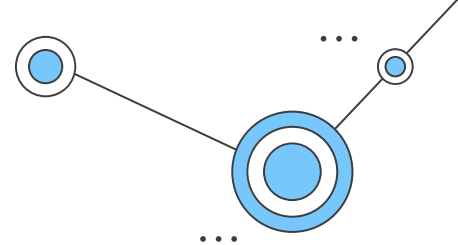
Lacunas funcionais;
Defeitos descobertos tardiamente.

Inconsistência

- Quando dois ou mais requisitos entram em conflito lógico.
- “R1: O sistema deve permitir acesso sem login.
R2: O sistema deve autenticar todos os usuários antes do acesso.”

Decisões arquiteturais comprometidas;
Bloqueio de desenvolvimento.

Falhas Recorrentes



A maioria das falhas em projetos de software não decorre de erro técnico de implementação, mas de problemas na definição, análise e gestão dos requisitos.

Excesso de Solução

- Quando o requisito já embute decisão técnica desnecessária.
- “O sistema deve utilizar banco de dados MySQL”

Não expressa necessidade;
Limita arquitetura;
Pode impedir otimizações.

Falta de Rastreabilidade

- Ausência de vínculo entre: requisito de negócio, requisito de usuário e casos de teste.

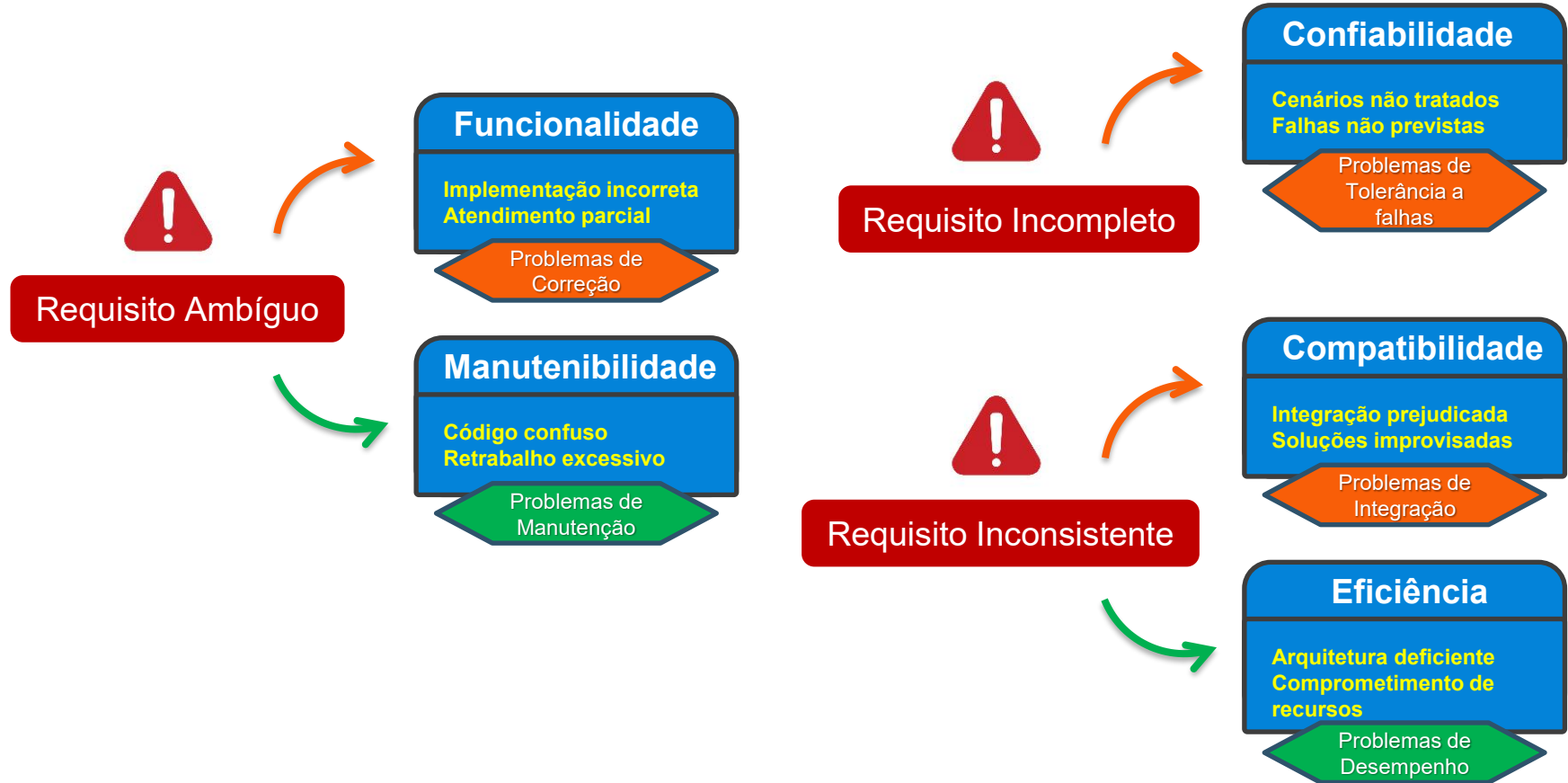
Dificuldade de
análise de impacto;
Mudanças caóticas.

Scope Creep

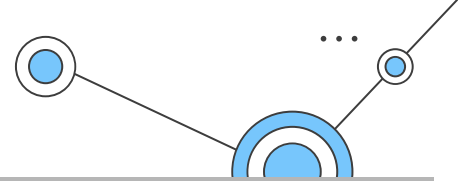
- Crescimento descontrolado do escopo sem controle formal de mudança.

Atraso;
Aumento de custo;
Perda de foco.

Falha × ISO 25010



Exercício



Em 26 de outubro de 1992, a London Ambulance Service (LAS) implantou um novo sistema informatizado de despacho de ambulâncias ("LASCAD") para agilizar o atendimento de chamadas de emergência. O software entrou em operação, mas falhou catastroficamente poucos dias após o lançamento, não suportando o volume de chamadas nem o fluxo de trabalho real dos despachantes. O sistema chegou a travar sob carga, gerando atrasos extremos no atendimento e interrupções na comunicação entre equipes no campo, obrigando o serviço a retornar ao processo manual de despacho.

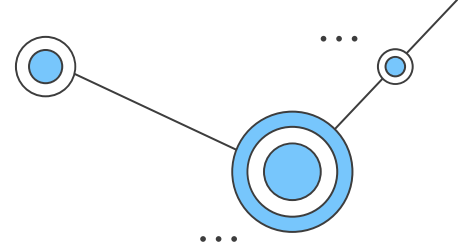
Estudos sobre esse caso indicam múltiplas causas: requisitos incompletos, falta de testes sob carga real, implementação apressada, pouco envolvimento dos usuários finais na definição das necessidades, falhas na especificação do sistema e treinamento inadequado dos operadores.

De fato, relatórios posteriores identificaram que o CAD (*Computer Aided Dispatch*) começou a apresentar problemas já nas primeiras horas de operação, inclusive devido à falta de testes de capacidade e a bugs conhecidos não corrigidos.

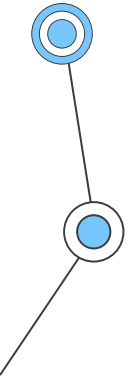
Esse caso é frequentemente citado como exemplo de como falhas na engenharia de requisitos, afetando de forma crítica atributos de qualidade de software.

Selecione dois atributos do modelo de qualidade ISO/IEC 25010 (ex: confiabilidade, eficiência, compatibilidade, usabilidade, portabilidade, manutenibilidade) que foram claramente comprometidos no caso LASCAD. Justifique sua escolha com exemplos do caso.

Referências



- BOEHM, Barry W. **A spiral model of software development and enhancement**. Computer, v. 21, n. 5, p. 61-72, May 1988. DOI: 10.1109/2.59.SOMMERVILLE, Ian. Software Engineering. 10. ed. Boston: Pearson, 2016.
- NAUR, Peter; RANDALL, Brian (Eds.). **Software Engineering: Report on a Conference Sponsored by the NATO Science Committee**. 1969.
- LAMSWEERDE, Axel van. **Requirements Engineering: From System Goals to UML Models to Software Specifications**. Chichester: John Wiley & Sons, 2009.
- WIEGERS, Karl; BEATTY, Joy. **Software Requirements**. 3. ed. Redmond: Microsoft Press, 2013.
- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION; INTERNATIONAL ELECTROTECHNICAL COMMISSION. ISO/IEC 25010:2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – **System and software quality models**. Geneva: ISO, 2011.
- BEYNON-DAVIES, Paul. **Human error and information systems failure: the case of the London ambulance service computer-aided despatch system project**. Interacting with Computers, v. 11, n. 6, p. 699-720, 1999. ISSN 0953-5438. DOI: [https://doi.org/10.1016/S0953-5438\(98\)00050-2](https://doi.org/10.1016/S0953-5438(98)00050-2).



Obrigado!

Prof. Me. Daniel Cunha da Silva

